CLASS 5

Cache in Our Django CRM

The exact files, code flow, live demo, and the next directions after this basic implementation.

CRM

TODAY

What did we add?

One cache type

Django **LocMemCache**: temporary memory inside Django.

One cached value

`customer_list`: the list shown on the customer page.

One visible proof

The browser displays whether the page used database or cache.

Goal: see a cache miss first, then a cache hit after refresh.

BEFORE CACHE

What did the customer page do before?

```
crm/registration/views.py

data = Registration.objects.all()
```



Every refresh repeated the database query, even if no customer data changed.

OUR DECISION

We chose local-memory application cache

LocMemCache

A built-in Django cache stored in this Django process's RAM.

Why for this class?

Simple setup, fast enough to demonstrate, and no new service to install.

This is an **application cache**, not a browser cache or CDN cache.

NO NEW PACKAGE

We used Django’s built-in cache support

```
from django.core.cache import cache
```

What we did not add

No Redis, Memcached server, Docker container, or package in `requirements.txt`.

Why that matters

Students can focus first on key, value, hit, miss, TTL, and clearing old data.

Django itself provides `cache.get()`, `cache.set()`, and `cache.delete()`.

CHANGE MAP

What changed, and where?

File	Change	Why
`crm/settings.py`	`CACHES` configuration	Choose LocMemCache and a 60-second TTL.
`registration/views.py`	Get/set/delete cache code	Reuse the customer list instead of repeatedly querying PostgreSQL.
`templates/customer.html`	“Cache demo” status message	Make hit and miss visible in the browser.

No model, migration, database-table, or external library change was needed.

SETTINGS

Tell Django which cache to use

```
crm/crm/settings.py

CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.locmem.LocMemCache',
        'LOCATION': 'crm-class-cache',
        'TIMEOUT': 60,
    }
}
```

BACKEND

The cache engine: local memory.

LOCATION

A name for this memory store.

TIMEOUT

Default lifetime: 60 seconds.

HOW DJANGO REACHES THE CODE

URL → view function → cache

```
crm/registration/urls.py

path('customer/', v.show_customer, name='customer')
```



The user does not write a cache key. The view code chooses the key.

THE KEY AND THE QUESTION

Where does Django ask cache?

```
crm/registration/views.py

cache_key = 'customer_list'
data = cache.get(cache_key)
```

Key

`customer_list` is the label for this cached customer-list value.

get()

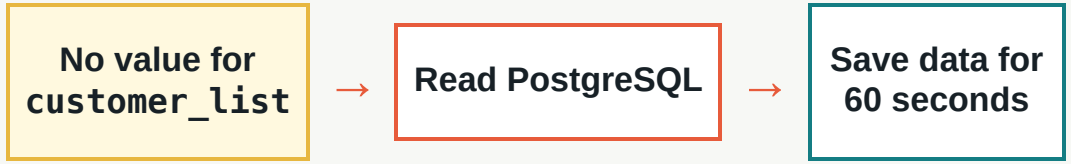
It asks: “Do you have a value saved with this exact key?”

Cache does not use intent detection here. It receives the exact key written by the developer.

CACHE MISS

If no saved value exists

```
if data is None:
    data = list(Registration.objects.values(
        'id', 'name', 'email', 'date_of_birth'
    ))
    cache.set(cache_key, data, timeout=60)
```



This is a **cache miss**. We save only safe display fields; the plain-text password field is not cached.

CACHE HIT

If cache already has the value

```
else:
    cache_status = 'Fast memory used: cache hit'
```



This is a **cache hit**: fast memory returns the temporary copy.

KEEP DATA FRESH

Clear cache after customer changes

```
cache.delete('customer_list')
```

Create

New customer added → old list is no longer complete.

Edit

Customer changed → cached name/email may be old.

Delete

Customer removed → cached list may still show them.

Deleting the old cache key makes the next request fetch and save fresh database data.

LIVE DEMO

What should students see?

Action	Browser message	What happened?
Open customer page first time	Database used: cache miss	Read PostgreSQL, then save cache.
Refresh within 60 seconds	Fast memory used: cache hit	Read the cached copy.
Wait 60 seconds and refresh	Database used: cache miss	TTL expired; save a fresh copy.
Edit or delete a customer	Database used: cache miss	`cache.delete()` cleared the old copy.

A Django restart also clears this local-memory cache. That is expected.

TRADE-OFF

Why not use this exact cache forever?

Good for today

- Built into Django
- No external service
- Fast and easy to understand

Limits

- Disappears after restart
- Each Django process has its own copy
- Not shared across multiple servers

Next production-level direction: a shared cache such as Redis or Memcached.

FUTURE DIRECTION

If we want to cache images

Do **not** put large uploaded image files inside today's small LocMemCache.

Store original image in media/object storage → **Create a small thumbnail** → **Use a stable image URL** → **Browser/CDN caches the image file**

When image changes

Use a new versioned URL, or clear/purge the CDN copy.

What our app could cache

Small image metadata, thumbnail URL, or repeated database result—not a huge original image.

FUTURE DIRECTION

Similarity search

Exact-key cache answers: “Do you have `customer_list`?”

Similarity search answers: “Which images, products, or documents are most similar to this one?”



A possible future PostgreSQL tool is **pgvector**. This is a separate feature from today’s simple cache, not a replacement for the database.

REMEMBER

Today’s CRM cache flow



- **Key:** `customer_list`
- **Value:** safe customer-list data
- **TTL:** 60 seconds
- **Freshness:** clear the key after create, edit, or delete

Sources: Django cache framework — docs.djangoproject.com/en/6.1/topics/cache/ | pgvector — github.com/pgvector/pgvector